

Universidade de Brasília
Departamento de Ciência da Computação
Teleinformática e Redes 1

Projeto Final

Gabriel Caixeta Romero - 232036896
Henrique Martins Fortes - 211061879
Pedro Conti - 221039245

1. Introdução

Este trabalho, desenvolvido na disciplina Teleinformática e Redes 1 do Departamento de Ciência da Computação da Universidade de Brasília (UnB), apresenta o Simulador TR1, um simulador de comunicação de dados implementado em Python. Uma interface de chat representa a troca de mensagens entre dois sistemas e permite acompanhar os dados da origem ao destino, passando pelas camadas de aplicação, enlace e física e por um meio com ruído. O usuário configura enquadramento, detecção e correção de erros, modulação e parâmetros do ruído, e a interface exibe os dados de cada etapa, os resultados de detecção e correção e os gráficos do sinal antes e depois do canal.

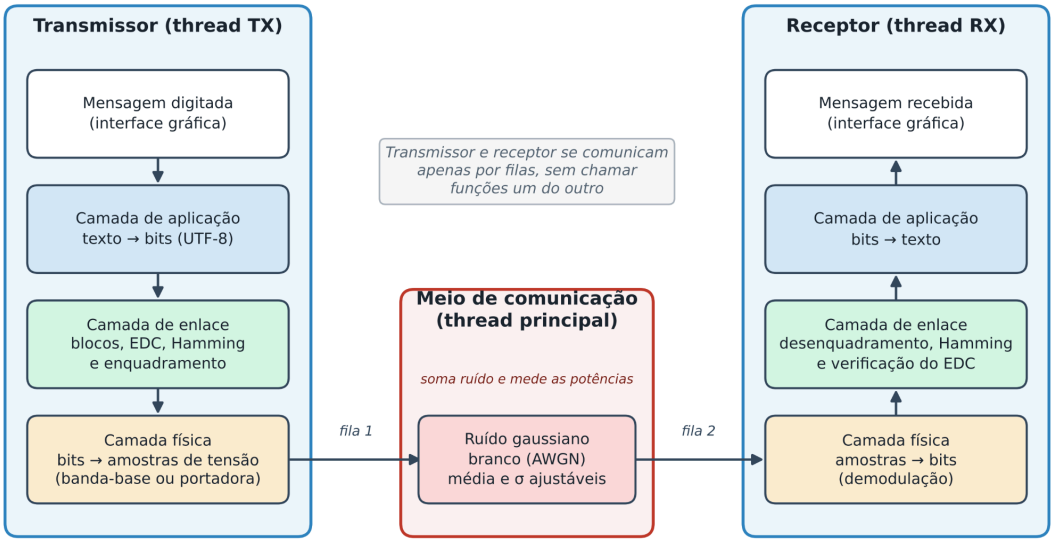
2. Implementação

O Simulador TR1 foi implementado em Python e dividido em seis módulos: camada de aplicação, camada de enlace, camada física, meio de comunicação, simulador e interface gráfica. Cada módulo implementa os protocolos de sua camada com funções próprias, sem bibliotecas externas que executem diretamente os algoritmos de enquadramento, modulação ou controle de erros.

2.1. Organização geral do simulador

O fluxo começa com a mensagem digitada na interface gráfica, convertida em bits, processada pela camada de enlace e encaminhada à camada física. O sinal atravessa então o meio de comunicação, que adiciona ruído. No receptor, as operações inversas recuperam os bits e reconstroem a mensagem.

Figura 1 – Fluxo geral de transmissão e recepção do simulador



Transmissor e receptor executam em threads próprias, enquanto a thread principal atua como meio de comunicação. A troca de dados ocorre por filas, o que evita que um módulo chame diretamente as funções do outro. O detalhamento está na Seção 2.6.

2.2. Camada de aplicação

O arquivo **`camada_aplicacao.py`** implementa texto_para_bits(), que codifica o texto em UTF-8 e gera os oito bits de cada byte, do mais significativo ao menos, e bits_para_texto(), que faz o caminho inverso. Na decodificação, bytes inválidos são substituídos por U+FFFD, de modo que o restante do texto seja exibido mesmo com bytes corrompidos.

2.3. Camada de enlace

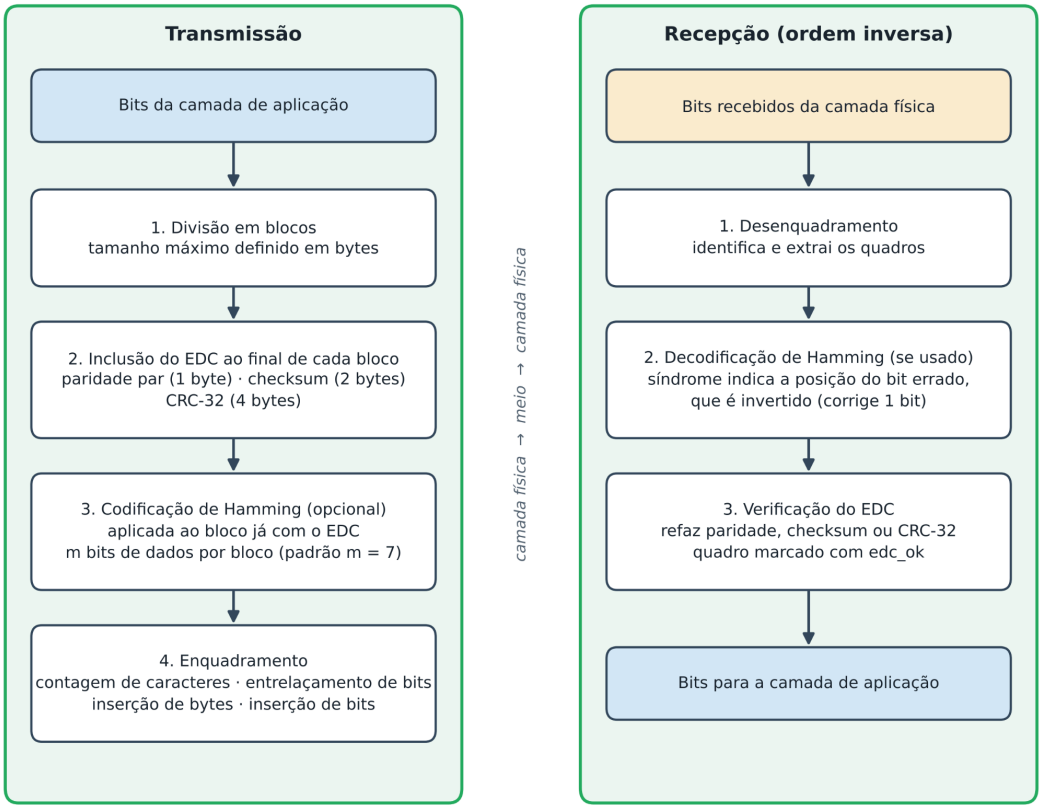
A camada de enlace foi implementada no arquivo **`camada_enlace.py`**. Sua função é preparar os dados recebidos da aplicação para a transmissão, dividindo a sequência de bits em blocos, acrescentando mecanismos de controle de erros e aplicando o método de enquadramento selecionado.

O processo de transmissão segue quatro passos, nesta ordem:

- Divisão dos bits da aplicação em blocos

- Inclusão do código de detecção de erros (EDC) ao final de cada bloco
- Codificação de Hamming, quando selecionada, aplicada ao bloco já acrescido do EDC
- Enquadramento dos blocos, que resulta em um único fluxo de bits enviado à camada física.

Figura 2 – Processamento dos dados na camada de enlace



No receptor, o procedimento é realizado na ordem inversa: primeiro os quadros são identificados e extraídos, depois o código de Hamming é decodificado, quando selecionado, e por último o mecanismo de detecção de erros verifica a integridade dos dados.

O usuário define o tamanho máximo dos blocos, em bytes. Cada bloco recebe seu próprio EDC e, quando habilitada, sua própria codificação de Hamming.

2.3.1. Métodos de enquadramento

Foram implementados quatro métodos de enquadramento: contagem de caracteres, entrelaçamento de bits, inserção de bytes e inserção de bits.

- **Contagem de caracteres**

Cada quadro começa com um byte com seu tamanho total, incluindo o próprio byte de contagem (dois bytes de conteúdo geram o valor 3). O receptor lê esse byte e extrai o conteúdo correspondente. O valor zero indica o fim do processamento, e o tamanho do quadro é limitado a 255 bytes pelo tamanho do cabeçalho.

- **Entrelaçamento de bits**

Os bits são distribuídos em uma matriz de oito colunas, e a paridade par de cada coluna, calculada por XOR, é acrescentada ao final do quadro como um byte. O quadro é delimitado por contagem de caracteres, o que permite ao receptor separar os dados da paridade. Na implementação atual, os bits de dados mantêm a ordem original (não há permutação); a função de desenquadramento calcula as colunas divergentes, mas essa informação não é repassada ao relatório de quadros.

- **Inserção de bytes (Byte Stuffing)**

Cada quadro é delimitado pelo byte FLAG (0x7E). Se o conteúdo contiver FLAG ou o byte de escape ESC (0x7D), o transmissor insere um ESC antes dele. O receptor identifica os delimitadores e, ao encontrar um ESC, trata o byte seguinte como dado e remove o escape.

- **Inserção de bits (Bit Stuffing)**

O delimitador é a sequência 01111110. O transmissor insere um bit 0 após cada cinco bits 1 consecutivos no conteúdo, e o receptor remove esse bit ao identificar o mesmo padrão.

2.3.2. Detecção de erros

Antes do enquadramento, cada bloco recebe ao final um código de detecção de erros (EDC) escolhido pelo usuário, sempre com número inteiro de bytes.

- **Bit de paridade par**

O EDC é um byte com sete zeros e um bit de paridade que torna par o total de bits 1 do bloco. O receptor refaz a contagem e sinaliza erro se o total for ímpar.

- **Checksum**

Soma em complemento de 1 de palavras de 16 bits (preenchidas com zeros, se necessário), com o vai um somado de volta ao resultado. O EDC, de 2 bytes, é o complemento da soma. O receptor soma todas as palavras, incluindo o checksum, e espera 0xFFFF, como no checksum da Internet.

- **CRC-32**

Divisão módulo 2 (XOR) bit a bit, sem bibliotecas como zlib, pelo gerador IEEE 802 (0x104C11DB7), sem valor inicial nem XOR final. O transmissor anexa 32 zeros ao bloco e usa o resto de 32 bits como EDC (4 bytes); o receptor divide o quadro inteiro e espera resto zero. Os testes verificaram a detecção de rajadas de até 32 bits.

Se o EDC não confere, o quadro não é descartado: ele segue marcado com edc_ok = False, e a interface exibe a falha e o texto possivelmente corrompido.

2.3.3. Correção de erros: código de Hamming

Os bits de paridade ocupam as posições que são potências de 2. Para m bits de dados, r é o menor valor com 2^r ≥ m + r + 1; o padrão é m = 7, que resulta no código (11, 7), e m é configurável na interface. O código é aplicado ao bloco já com o EDC, que fica assim protegido também. No receptor, a síndrome indica a posição do bit errado, que é invertido; erros duplos não são corrigidos, mas o EDC os revela. Para que o receptor identifique o fim dos dados com qualquer m, acrescenta-se um bit 1 seguido de zeros até um múltiplo de m, e a saída é completada com zeros até um múltiplo de 8 bits.

2.4. Camada física

A camada física converte bits em amostras de tensão (Volts) e faz o caminho inverso. Cada bit (banda-base) ou símbolo (portadora) é representado por 100 amostras, com amplitude de referência V = 1,0 V.

2.4.1. Modulação banda-base

- **NRZ-Polar:** bit 1 = $+V$ e bit 0 = $-V$ durante todo o bit. A decisão usa a média das 100 amostras (positiva = 1).

- **Manchester:** cada bit tem uma transição no meio: bit 1 = alto→baixo (+V na primeira metade e −V na segunda) e bit 0 = baixo→alto, o que equivale a dado XOR relógio. A decisão compara a média das duas metades do bit.
- **Bipolar (AMI):** bit 0 = 0 V e cada bit 1 alterna entre +V e −V. O demodulador classifica o bit como 1 quando o módulo da média excede V/2.

A média de todas as amostras, em vez da amostra central, aproveita as 100 amostras e atenua o ruído gaussiano.

2.4.2. Modulação por portadora

Na modulação por portadora, os bits são agrupados em símbolos (o último é completado com zeros, ignorados no desenquadramento). O demodulador correlaciona cada símbolo com cos e −sen, obtém I e Q e escolhe o ponto de constelação mais próximo. As modulações implementadas são:

- **ASK (on-off):** bit 1 = portadora de amplitude V e bit 0 = 0 V. O limiar é V/2 sobre $\sqrt{(I^2 + Q^2)}$.
- **FSK:** duas frequências, de 2 e 4 ciclos por símbolo, com a mesma amplitude. Por serem inteiras no símbolo, são ortogonais e a correlação é exata; o demodulador escolhe a frequência de maior energia.
- **8PSK:** 3 bits por símbolo, fase $k \cdot 45°$ e raio V. Os rótulos seguem código Gray (000, 001, 011, 010, 110, 111, 101, 100), de modo que um erro para a fase vizinha corrompe um único bit.
- **32-QAM:** constelação em cruz de 6×6 pontos sem os 4 cantos (os de maior energia), com níveis ±1, ±3 e ±5 multiplicados por V/5. Como 32 não é quadrado perfeito, não existe código Gray perfeito: adotaram-se 2 bits de quadrante (Gray) e 3 bits de posição ao longo de um ciclo de 8 pontos do quadrante, de modo que 48 dos 52 pares de vizinhos diferem em 1 bit.

2.5. Meio de comunicação

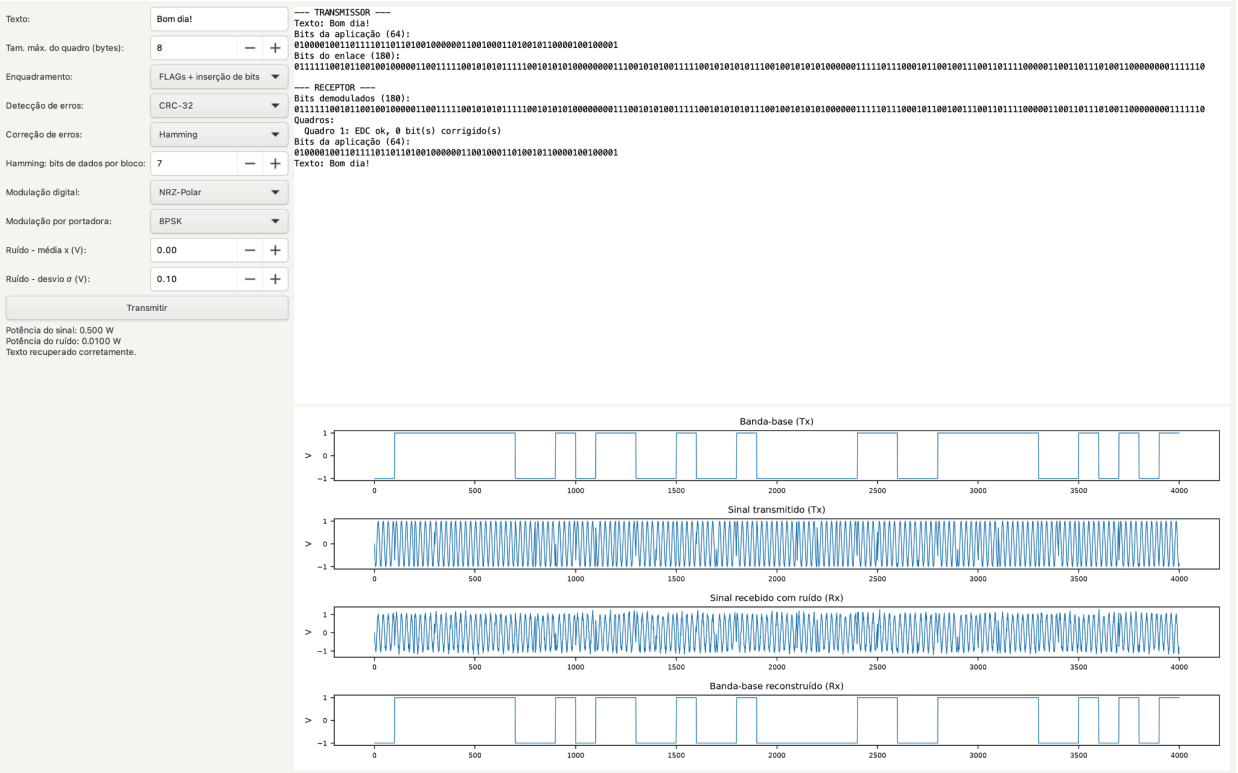
O meio soma a cada amostra um ruído gaussiano branco (AWGN) com média e desvio padrão σ configuráveis, gerado com random.gauss, e calcula a potência média do sinal e do ruído, P = média(v^2), em Watts sobre carga de 1 Ω. A interface exibe as duas potências.

2.6. Simulador, threads e interface gráfica

O módulo `simulador.py` cria duas threads (`threading.Thread`). A thread de transmissão (TX) aplica as camadas de aplicação, enlace e física e coloca o sinal em uma fila. A thread principal atua como meio de comunicação e adiciona o ruído. A thread de recepção (RX) retira o sinal da segunda fila e executa as operações na ordem inversa. Erros ocorridos nas threads são capturados e relançados na thread principal.

A interface (interface_gui.py, GTK 3 com matplotlib) apenas lê os campos, monta o dicionário de configuração, chama o simulador e exibe os bits de cada etapa, o resultado por quadro (erros corrigidos e EDC válido), as potências e os gráficos do sinal antes e depois do canal.

Figura 3 – Interface gráfica do simulador: configuração, saída do transmissor e do receptor e gráficos do sinal



3. Testes

O arquivo testes.py executa, sem GTK, testes de ida e volta (a saída do receptor deve igualar a entrada do transmissor) para o meio, a aplicação, os quatro enquadramentos (incluindo casos críticos de stuffing e padding), os três EDCs (incluindo rajada de 32 bits no CRC), o Hamming (sem erro e com erro em cada posição), as constelações, todas as

modulações com e sem ruído e a simulação completa (4 enquadramentos × 5 opções de portadora). Os 64 testes são aprovados.

4. Execução

A instalação das dependências e a execução do simulador e dos testes seguem os comandos abaixo:

```
sudo apt install python3-gi python3-gi-cairo gir1.2-gtk-3.0
python3 -m venv .venv --system-site-packages
source .venv/bin/activate
pip install -r requirements.txt
python3 simulador.py      # interface gráfica
python3 testes.py         # testes
```

5. Conclusão

O Simulador TR1 integra as camadas de aplicação, enlace e física e o meio de comunicação em uma única aplicação, com os protocolos exigidos implementados sem bibliotecas externas. O simulador permitiu observar o efeito do ruído sobre o sinal, a delimitação dos quadros por cada método de enquadramento, a detecção de falhas pelos EDCs e a correção de erros simples pelo código de Hamming.

Observou-se que, com σ elevado, a modulação 32-QAM falha antes da 8PSK, porque os pontos de sua constelação ficam mais próximos entre si. Observou-se também que o CRC detecta erros que a paridade deixa passar, como a inversão de um número par de bits.